



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A KOMUNIKAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV TELEKOMUNIKACÍ

DEPARTMENT OF TELECOMMUNICATIONS

MULTIPLATFORMNÍ APLIKACE PRO HROMADNOU SPRÁVU ZAŘÍZENÍ MIKROTIK ROUTEROS

MULTI-PLATFORM APPLICATION FOR BULK MANAGEMENT OF MIKROTIK ROUTEROS DEVICES

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

Jiří Kozárek

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. Ondřej Krajsa, Ph.D.

BRNO 2026

Bakalářská práce

bakalářský studijní program **Telekomunikační a informační systémy**

Ústav telekomunikací

Student: Jiří Kozárek

ID: 256766

Ročník: 3

Akademický rok: 2025/26

NÁZEV TÉMATU:

Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS

POKYNY PRO VYPRACOVÁNÍ:

Navrhněte a implementujte multiplatformní aplikaci (Windows, Linux) v Pythonu pro správu zařízení s RouterOS prostřednictvím oficiálního API (šifrované připojení TLS na portu 8729). Aplikace musí umožnit kompletní správu jednotlivých zařízení, bezpečné uložení přístupových údajů, jednotlivé i hromadné provádění konfiguračních změn (např. přes šablony), zálohu a obnovu konfigurace a jednoduchou správu logů. Výstupem práce bude plně funkční aplikace, zdrojové kódy a uživatelská a instalační příručka.

DOPORUČENÁ LITERATURA:

Podle pokynů vedoucího práce.

Termín zadání: 9.2.2026

Termín odevzdání: 2.6.2026

Vedoucí práce: Ing. Ondřej Krajsa, Ph.D.

prof. Ing. Jiří Mišurec, CSc.
předseda rady studijního programu

UPOZORNĚNÍ:

Autor bakalářské práce nesmí při vytváření bakalářské práce porušit autorská práva třetích osob, zejména nesmí zasahovat nedovoleným způsobem do cizích autorských práv osobnostních a musí si být plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

ABSTRAKT

Bakalářská práce má za cíl vytvořit multiplatformní aplikaci v programovacím jazyce Python, jež bude schopna provádět hromadné konfigurace aktivních síťových prvků, na kterých běží operační systém MikroTik RouterOS. Konfigurace bude probíhat z grafického rozhraní pomocí systémového rozhraní API na portu 8729, konkrétně jeho šifrované verze zabezpečené pomocí TLS. Celá aplikace je dostupná v repozitáři na platformě GitHub.

KLÍČOVÁ SLOVA

Python, MikroTik, RouterOS, hromadná konfigurace

ABSTRACT

The goal of this bachelor's thesis is creation of cross-platform application written in Python programming language that will be capable of mass configuration of active network devices running MikroTik RouterOS operating system. The configuration will be done via graphical user interface using system API interface on port 8729, specifically its encrypted version secured by TLS. The entire app is available in a repository on GitHub platform.

KEYWORDS

Python, MikroTik, RouterOS, mass configuration

KOZÁREK, Jiří. *Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS*. Bakalářská práce. Brno: Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav telekomunikací, 2026. Vedoucí práce: Ing. Ondřej Krajsa, Ph.D.

Prohlášení autora o původnosti díla

Jméno a příjmení autora:	Jiří Kozárek
VUT ID autora:	256766
Typ práce:	Bakalářská práce
Akademický rok:	2025/26
Téma závěrečné práce:	Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS

Prohlašuji, že svou závěrečnou práci jsem vypracoval samostatně pod vedením vedoucí/ho závěrečné práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury na konci práce. V souvislosti s vytvořením závěrečné práce jsem neporušil zásady a doporučení VUT k využívání generativní umělé inteligence.

Jako autor uvedené závěrečné práce dále prohlašuji, že v souvislosti s vytvořením této závěrečné práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a/nebo majetkových a jsem si plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

Brno
.....
podpis autora*

*Autor podepisuje pouze v tištěné verzi.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu bakalářské práce panu Ing. Ondřeji Krajsovi, Ph.D. za vedení, konzultace a podnětné návrhy k práci. Dále bych chtěl také poděkovat panu Bc. Jakubu Raimrovi za rady při psaní této práce a panu Martinu Smejkalovi za věcné rady v oblasti zabezpečení dat, bezpečnosti a programování.

Obsah

Úvod	10
1 Síťová zařízení	11
1.1 Prvky sítě	11
1.2 MikroTik	11
2 RouterOS	13
2.1 Konfigurace	13
2.2 winbox/webfig	13
2.3 API	15
3 Databáze	16
3.1 Možné realizace databáze	16
3.1.1 SQLite	16
3.2 Ukládání hesel	17
4 Grafické rozhraní	18
5 Pohledy	19
6 Správa logů	19
7 Struktura aplikace	20
7.1 Úvodní aplikace	20
7.1.1 Vnitřní struktura	21
7.2 Konfigurační aplikace	22
7.2.1 Vnitřní struktura	24
Závěr	28
Literatura	29
Seznam symbolů a zkratk	31
Seznam příloh	32

Seznam obrázků

1.1	Ukázka routeru společnosti MikroTik	12
2.1	Některé z možností konfigurace ROS	13
2.2	Ukázka grafického rozhraní WinBox	14
2.3	Ukázka webového rozhraní WebFig	14
4.1	Grafické rozhraní konfiguračního okna (vývojová verze)	18
7.1	Grafické rozhraní hlavní aplikace	20
7.2	Grafické rozhraní konfigurační aplikace	23
7.3	Detail grafického rozhraní konfigurační aplikace - sekce <i>common</i> . . .	24
7.4	Detail grafického rozhraní konfigurační aplikace - záznamy vybraného zařízení	24
7.5	Detail grafického rozhraní konfigurační aplikace - konfigurační podokno	25

Seznam výpisů

2.1	Ukázka komunikace pomocí api s využitím ukázkového skriptu [11], zkráceno	15
7.1	Metoda objektu <i>db_crypto</i> (7.1.1) pro zašifrování a uložení přístupových údajů	22
7.2	Metoda objektu <i>middleware</i> (7.2.1) sloužící k provedení konfiguračních změn	26

Úvod

Úvod studentské práce, např. . .

Nečíslovaná kapitola Úvod obsahuje „seznámení“ čtenáře s problematikou práce. Typicky se zde uvádí: (a) do jaké tematické oblasti práce spadá, (b) co jsou hlavní cíle celé práce a (c) jakým způsobem jich bylo dosaženo. Úvod zpravidla nepřesahuje jednu stranu. Poslední odstavec Úvodu standardně představuje základní strukturu celého dokumentu.

Tato práce se věnuje oblasti DSP (číslicové zpracování signálů – Digital Signal Processing), zejména jevům, které nastanou při nedodržení Nyquistovy podmínky pro *vzorkovací kmitočet* (f_{vz}).¹

Šablona je nastavena na *dvoustranný tisk*. Nebudte překvapeni, že ve vzniklém PDF jsou volné stránky. Je to proto, aby důležité stránky jako např. začátky kapitol začínaly po vytisknutí a svázání vždy na pravé straně. Pokud máte nějaký závažný důvod sázet (a zejména tisknout) jednostranně, nezapomeňte si přepnout volbu `twoside` na `oneside`!

¹Tato věta je pouze ukázkou použití příkazů pro sazbu zkratk.

1 Síťová zařízení

Dnešní svět, silně propojený celosvětovou počítačovou sítí Internet, si bez její existence již nedokážeme představit. Díky této síti jsme schopni docílit téměř real-time komunikace, a to z kteréhokoliv místa na světě na jiné. Vznik takovéto sítě je podmíněn vysokým stupněm standardizace [1].

1.1 Prvky sítě

Obecně se počítačové sítě skládají ze dvou základních typů prvků, a to pasivních a aktivních.

Mezi pasivní prvky řadíme ty části sítě, které nezasahují do datového provozu a nemají interpretovat význam přenášených signálů. Jedná se například o kabely, spojky, konektory. Tyto prvky po jejich instalaci nevyžadují údržbu.

Naopak aktivní prvky jsou prvky, které do jisté míry dokáží interpretovat přenášené signály. Dle typu aktivního prvku jsou tyto informace využívány k přepínání, směrování či dokonce může docházet i k zásahu do daného datového provozu, zahazování či změně parametrů.

Takovéto prvky je většinou nutné po zapnutí nastavit, aby vykonávaly požadované úkony. Dále je také nezbytné, aby byly prvky aktivně spravovány, tzn. docházelo k úpravám konfigurace dle aktuálních potřeb dané sítě či bezpečnostnímu záplatování.

O správu aktivních prvků se často starají specialisté nazývaní jako správci sítě.

1.2 MikroTik

Na světě existuje nespočet firem, malých či velkých, které se zabývají vývojem nebo výrobou speciálního softwaru či hardwaru pro síťové aplikace. Namátkou lze vybrat firmy známé, jejichž jména se stala v oboru ikonickými, jako jsou společnosti CISCO Systems [2], HP (Hewlett-Packard) [3] a její odnož Enterprise [4], či Arista [5]. Z řady menších, ale stále dosti významných firem, stojí za zmínku firma Ubiquiti [6], která se snaží o zjednodušení konfigurace a správy aktivních síťových prvků.

Zaměřením této práce je však litevská firma SIA Mikrotīks, známá jako MikroTik [7]. Jedná se o výrobce kompletních síťových řešení v oblasti aktivních prvků ve smyslu výroby vlastního síťového hardwaru [8] a především síťového softwaru RouterOS [9].



Obr. 1.1: Ukázka routeru společnosti MikroTik

2 RouterOS

Síťový operační systém ROS společnosti MikroTik je komunitou vysoce oblíbený a rozšířený napříč spektrem síťových zařízení. Jeho distribuce probíhá společně s hardwarovými produkty. Díky oblibě komunity se také stal samostatným produktem společnosti. Lze jej pořídit samostatně a provozovat přímo na strojích s architekturou x86 či využít verzi CHR a provozovat tento systém jako virtuální stroj. Tímto operačním systémem je nabízena konfigurace nepřeberného množství protokolů převážně na prvních třech vrstvách ISO/OSI modelu [10].

2.1 Konfigurace

Systémem ROS je nabízeno velké množství způsobů konfigurace. K navigaci je použita stromová struktura příkazů, ve které je třeba se pro podrobnější správu vydat po dané větvi dál od konfiguračního kmene až k samotným možným argumentům příkazů. Na následujícím obrázku je zobrazen seznam možností přístupu konfigurace a port, na kterém služba aktuálně naslouchá. Stojí za zmínku, že všechny zobrazené služby běží na standardních (výchozích) portech.

Name	Port
api	8728
api-ssl	8729
ftp	21
ssh	22
telnet	23
winbox	8291
www	80
www-ssl	443

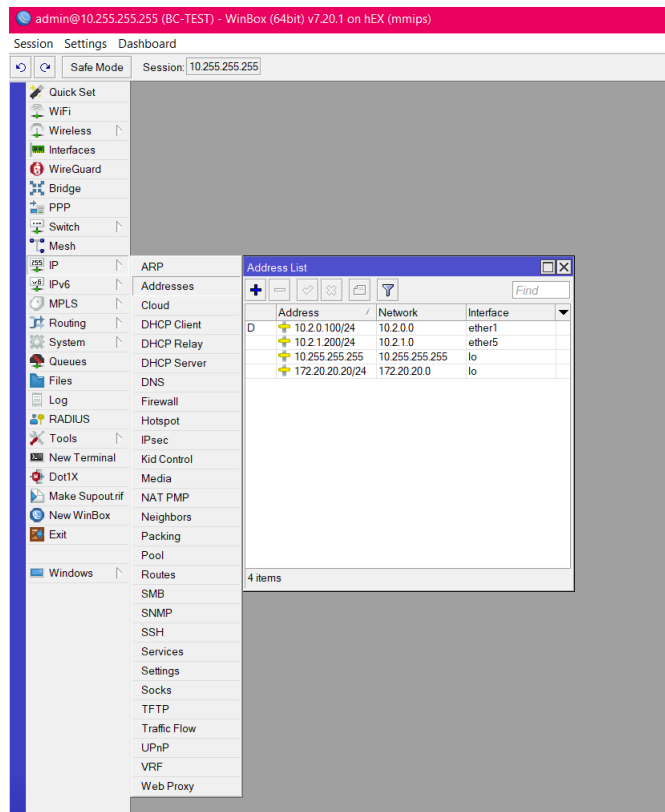
Obr. 2.1: Některé z možností konfigurace ROS

Pro účely této práce jsou však jiné způsoby konfigurace nežli *winbox/webfig* a *api* bezvýznamné, proto jim již nebude v této práci věnována větší pozornost.

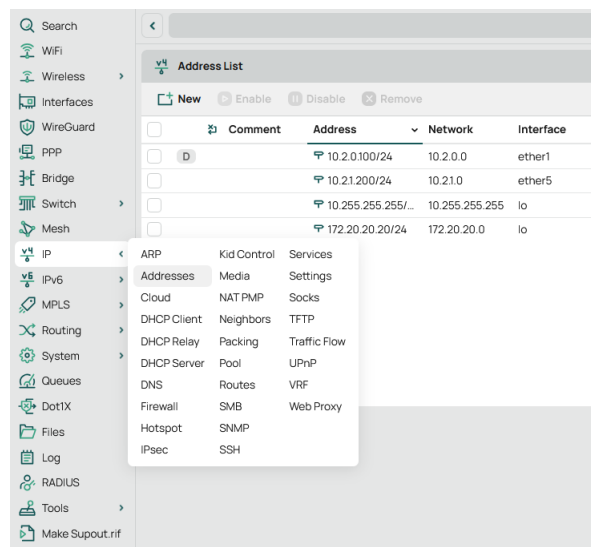
2.2 winbox/webfig

WinBox je grafický proprietární konfigurační software pro správu zařízení se systémem Mikrotik RouterOS. Je založen na systému oken s konfiguračními podokny, pomocí kterých se správa provádí. Lze konstatovat, že se jedná o grafické zobrazení stromové hierarchie příkazů dostupných v konzoli. Stejně tak lze konstatovat, že konfigurační rozhraní webfig plní funkci winboxu. Jedná se však o webovou aplikaci vzhledově a do jisté míry i funkčně podobnou aplikaci winbox. Její největší výhoda

spočívá v možnosti konfigurace prakticky z každého zařízení, na kterém lze spustit webový prohlížeč.



Obr. 2.2: Ukázka grafického rozhraní WinBox



Obr. 2.3: Ukázka webového rozhraní WebFig

2.3 API

Jedná se o přímé napojení využívající komunikaci na úrovni bytů, kterou je třeba dekodovat. Samotná užitečná komunikace pak probíhá ve formátu příkazového slova společně s jeho argumenty, následuje odpověď daného zařízení ve formě výčtu ve formátu parametr – hodnota.

Výpis 2.1: Ukázka komunikace pomocí api s využitím ukázkového skriptu [11], zkráceno

```
<<< /ip/address/print
<<<
>>> !re
>>> =.id=*2
>>> =address=10.255.255.255/32
>>> =network=10.255.255.255
>>> =interface=lo
>>> =actual-interface=lo
>>> =invalid=false
>>> =dynamic=false
>>> =slave=false
>>> =disabled=false
```

Ukázka nevyužívá všechna možná slova a parametry, které lze použít při komunikaci. Pro ukázkou je tento jednoduchý příkaz dostatečný. Dále je také v práci uvažována komunikace s api rozhraním v zabezpečené formě využívající TLS - v ukázce pod názvem *api-ssl*.

3 Databáze

Jedním ze základních úkolů této práce pro autora byl návrh databáze a systému ukládání hesel. Struktura databáze má dalekosáhlé důsledky pro návrh celé architektury aplikace a jejím budoucím možnostem.

3.1 Možné realizace databáze

Autor vybíral ze základních dvou přístupů pro práci s databází. Jedním z nich je přístup pomocí serveru. V tomto modelu je třeba serverová část, spuštěná lokálně či na vzdáleném serveru, ke které se uživatel připojí a využívá její služby pomocí síťového stacku operačního systému. Má však velké úskalí, díky kterému je dle autora nevhodná pro tuto aplikaci.

Při lokálně spuštěné serverové instanci je nejprve nutno tuto instanci na daném počítači nainstalovat a spustit. Takové řešení sice je možné, ale autor má za to, že takové řešení přidává zbytečnou komplexitu projektu.

Při použití vzdáleného serveru je předpokládána funkčnost síťového spojení na daný server a jeho funkčnost. Z logiky vyplývá, že by zde mohl nastat zásadní problém, a to v momentu výpadku. Nastala by situace, kdy by síťové prvky, nutné pro spojení s databázovým serverem, nemohly být nakonfigurovány, jelikož přístupové údaje k těmto prvkům by byly uloženy na v tu chvíli nedostupném serveru. Z výše uvedeného vyplývají zásadní možné chyby a problémy dané realizace databáze, proto autor vybral alternativní metodu uvedenou níže.

3.1.1 SQLite

Díky předešlým úvahám a vlastnostem uvedených systémů autor využil v aplikaci SQLite [12], konkrétně implementaci tohoto systému knihovnou sqlite3 [13]. Jedná se o lightweight knihovnu obsaženou přímo v pythonu, není tedy třeba její doinstalace. Tento systém nevyužívá spojení na databázový server, nýbrž lokální databázový soubor. Aplikace tedy bude fungovat i v případě kompletního odpojení od sítě či jejím výpadku. Je třeba zmínit hlavní nevýhodu tohoto přístupu, a toum je nutnost databázové soubory distribuovat či synchronizovat. Dále také fakt, že nemůže být přístupováno k databázovému souboru vícero uživateli najednou. Při využívání databázového souboru je soubor zamčen pro čtení i zápis ostatním uživatelům. Potenciální problémy s přístupem k databázovému souboru však nejsou pro tuto aplikaci relevantní, jelikož aplikaci může využívat v daném okamžiku na jednom počítači pouze jeden uživatel. Otázka distribuce a synchronizace bude popsána dále.

Proto má autor za to, že řešení pomocí systému SQLite, používající lokální databázové soubory, bude nejvhodnějším řešením.

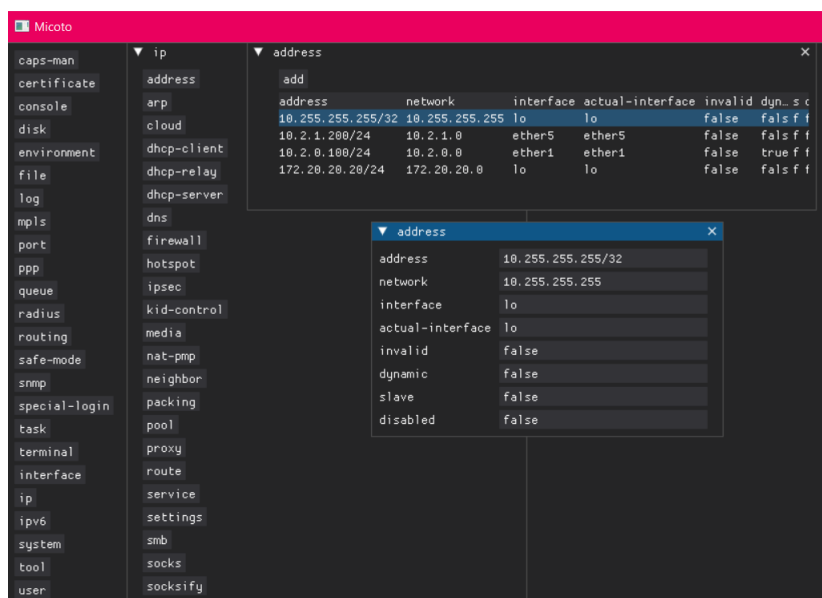
3.2 Ukládání hesel

Ačkoli je autorovi známo, že ukládání hesel jako takových je naprosto nevhodné a silně nedoporučené, v tomto případě není zbytlí. Systém ROS nepodporuje pro přístup pomocí api jinou autentizační metodu nežli přihlášení uživatelským jménem a heslem. Jelikož je jeden pohled aplikace zamýšlen spíše uživatelsky, viz dále, je nutné vytvořit jednoduchý systém, kdy po zadání jednoho hesla budou zpřístupněna všechna zařízení. Protože nastavení stejného hesla do vícero zařízení není z bezpečnostního hlediska lepším řešením, zvolil autor řešení využívající databázi se zašifrovanými přístupovými hesly.

Prvně bylo autorem zamýšleno využít systém symetrického šifrování s využitím knihovny cryptography, a to konkrétně již připraveným systémem Fernet [14]. Tento systém je jednoduchý na implementaci, avšak po konzultaci s vedoucím práce bylo rozhodnuto pro jeho nevyužití, jelikož pro šifrování je využíván AES-128-CBC. Ačkoli je AES-128 dnes stále považován za bezpečný a splňuje minimální požadavky NÚKIBu [15], bylo rozhodnuto o nevyužití tohoto systému a implementaci řešení využívající AES-256-CBC, na který je aktuálně nahlíženo jako i do budoucna bezpečný algoritmus.

4 Grafické rozhraní

Práce s aplikací má probíhat pomocí GUI. Programovací jazyk python nabízí nemalé množství knihoven, které implementaci grafických rozhraní ulehčují [16]. Jelikož je předpoklad funkční podobnosti aplikace s aplikací WinBox, bylo rozhodnuto využít knihovnu DearPyGui [17] [18]. Jedná se o lightweight grafickou knihovnu s hardwarovou akcelerací. Hlavním důvodem výběru této knihovny je možnost jednoduché práce s okny v okně. Předpoklad této funkce je stěžejní pro celou koncepci programu.



Obr. 4.1: Grafické rozhraní konfiguračního okna (vývojová verze)

5 Pohledy

Spousta systémů síťových prvků nabízí možnost využít tzv. „views“ či „role-based“ přístup [19]. Tato funkcionality umožňuje definovat, k jakým příkazům a jejich parametrům bude mít daný uživatel systému přístup. Právě tato funkcionality v systému ROS chybí. Díky výše zmíněné volbě architektury (3) využívající lokální databázi dostupných příkazů lze vytvořit pro daného uživatele speciální databázový soubor obsahující pouze příkazy a parametry, které potřebuje ke své práci, a to za účelem zjednodušení či jako „bezpečnostní“ faktor, kdy není uživatel schopen jednoduše procházet či měnit části konfigurace z aplikace, s jejichž fungováním není plně seznámen a/nebo k nim nemá mít přístup.

6 Správa logů

Po rešerši na téma správy logů bylo po dohodě s vedoucím rozhodnuto, že tuto oblast aplikace Micoto nebude pokrývat.

Jedním z hlavních důvodů jsou již existující centralizované systémy, a to už komerční či open source, mezi které patří například Zabbix [20] či LibreNMS [21].

7 Struktura aplikace

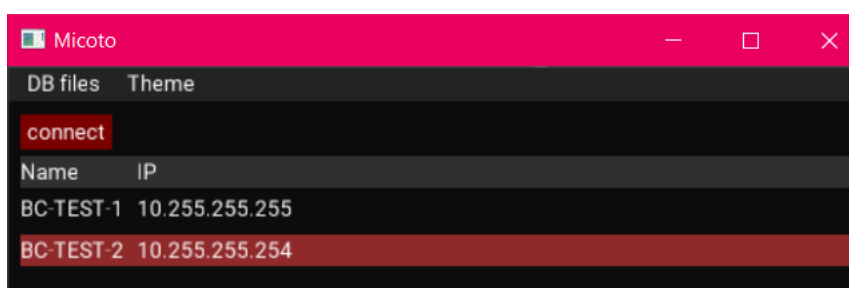
Aplikace je rozdělena do několika samostatně funkčních celků, a to administrátorské aplikace, samotné konfigurační aplikace a úvodní aplikace, která je otevřena ihned po spuštění aplikace Micoto.

Co se databází týče, je aplikace rozdělena na dvě části. Tyto části jsou databáze s příkazy a databáze s přihlašovacími údaji. Toto rozdělení má své výhody i nevýhody. Pro dané rozdělení databází bylo rozhodnuto kvůli praktičnosti reálného nasazení. Administrátorovi stačí pro změnu dostupných příkazů či změně dostupných zařízení redistribuovat nové databázové soubory s definovanou strukturou, a to nezávisle na ostatních uživateli. Tato skutečnost také umožňuje tyto dvě databáze upravovat nezávisle na sobě a tvořit jejich kombinace.

7.1 Úvodní aplikace

Tato aplikace je základním prvkem pro nastavení a ovládání aplikace. Nabízí jednoduché uživatelské rozhraní.

Mezi prvotní úkony v této aplikaci je výběr databázového souboru obsahujícího přístupové údaje ke spravovaným zařízením a zadáním hesla pro rozšifrování těchto údajů. Jako další krok je výběr a nastavení databázového souboru obsahujícího sadu dostupných příkazů. Následně lze údaje rozšifrovat. Dostupná zařízení z rozšifrované databáze jsou zobrazena v okně. Kliknutím na daný záznam dojde k jeho označení. Po výběru požadovaných zařízení lze kliknutím na tlačítko *connect* provést spojení s danými zařízeními. Dojde k otevření nového okna, a to okna aplikace sloužící ke konfiguraci dříve vybraných zařízení.



Obr. 7.1: Grafické rozhraní hlavní aplikace

7.1.1 Vnitřní struktura

Aplikace se spouští buďto příkazem či poklepaním na soubor. Pokud chce daný uživatel spustit aplikaci, není třeba aplikaci spouštět s parametry. Aplikaci lze také spustit v administrátorském módu, pro jehož spuštění je třeba vytvořit objekt typu *GUI* s parametrem *admin=True*. Nastavením hodnoty parametru *admin* na *True* způsobí zobrazení jinak skrytých grafických prvků, kterými lze aktivovat administrátorské části kódu, které jsou jinak uživateli skryty.

Třída *db_crypto*

Tato třída reprezentuje a realizuje operace spojené s vytvořením a úpravami databáze zařízení. Společně s temito operacemi také zodpovídá za šifrování a dešifrování přístupových údajů. Mimo jiné využívá knihovnu *cryptography*.

Výpis 7.1: Metoda objektu *db_crypto* (7.1.1) pro zašifrování a uložení přístupových údajů

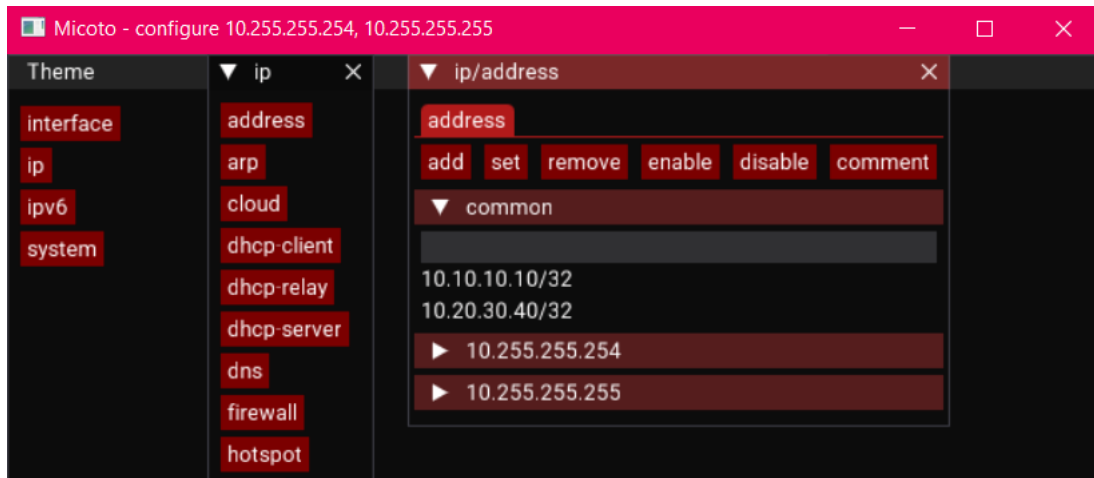
```
1  def insert(self, devIp, devUser, devPass):
2      """Inserts new device to device database, catches
3          and saves system name"""
4      # ověří, zda zařízení se stejnou IP již není v
5          databázi
6      if self.checkExistance(devIp):
7          return False
8      #devHost
9      # pokusí se navázat spojení se zařízením pro získá
10         ní hostname
11      identity = apiros.getResponse(devIp, devUser,
12          devPass, "/system/identity/print")
13      if identity:
14          devHost = identity["name"]
15      else:
16          devHost = ""
17      #db insert
18      # pošle vložené heslo do funkce vracející zaš
19         ifrované heslo (ciphertext) a iv (
20             initialization vector)
21      devCipher, devIv = self.encrypt(devPass.encode().
22          ljust(32)[:32])
23      # uložení dat do databáze
24      self.cur.execute("INSERT INTO devices (devHost,
25          devIp, devUser, devPass, devIv) VALUES (?, ?, ?,
26              ?, ?)", (devHost, devIp, devUser, devCipher,
27                  devIv))
28      self.con.commit()
29      return True
```

7.2 Konfigurační aplikace

Po inicializaci a připojení dle 7.1 k vybraným zařízením dojde k otevření okna konfigurační aplikace.

Na levé straně okna aplikace nabízí dostupné kořenové volby (příkazy), které má uživatel k dispozici. Klinutím na ně je otevřeno další vnitřní okno, které nabízí vý-

běr možných konfigurací a/nebo výpis již nastavených/dostupných prvků, na které je možné dané příkazy aplikovat.



Obr. 7.2: Grafické rozhraní konfigurační aplikace

Na obrázku 7.2 lze spatřit část okna s názvem *common*. Právě tato část je tou částí, kvůli které celá práce vznikla. Nachází se v ní společné prvky alespoň dvou zařízení, ke kterým je aktuálně konfigurační aplikace připojena. Výčet těchto zařízení se nachází v názvu okna, v tomto případě se jedná o zařízení s IP adresami 10.255.255.254 a 10.255.255.254.

Po přesunu kurzoru nad daný záznam je zobrazeno informační vyskakovací pole, ve kterém jsou uvedeny detaily daného záznamu pro dané zařízení. V tomto případě se jedná o IP adresu a informace k jejímu záznamu patřící, viz 7.3.

Pro zobrazení detailu všech nastavených parametrů pro záznamy stejného typu se nachází pod sekci *common* rolovací tabulka nesoucí IP adresu daného zařízení jako název. Po kliknutí na daný záznam je zobrazena daná tabulka 7.4.

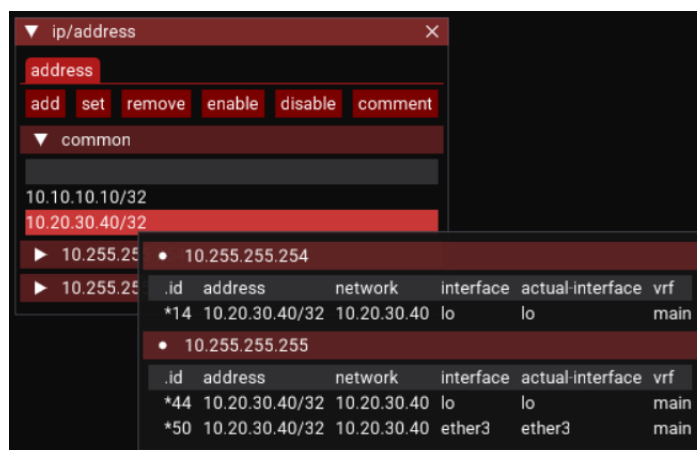
Práce s vybranými záznamy probíhá za použití tlačítek v horní části vnitřního konfiguračního okna. Po označení požadovaných záznamů a kliknutí na tlačítko s požadovanou akcí je zobrazeno konfigurační podokno se všemi možnostmi úprav. Pole, která mají u pravé strany šipku, po rozkliknutí zobrazí roletové menu, ve kterém se nacházejí dostupné možnosti konfigurace. Ostatní položky jsou text boxy určené k manuálnímu zadání hodnoty parametru, viz 7.5

Položka *numbers* zobrazuje seznam dříve vybraných zařízení pro konfiguraci včetně *id* jednotlivých upravovaných položek.

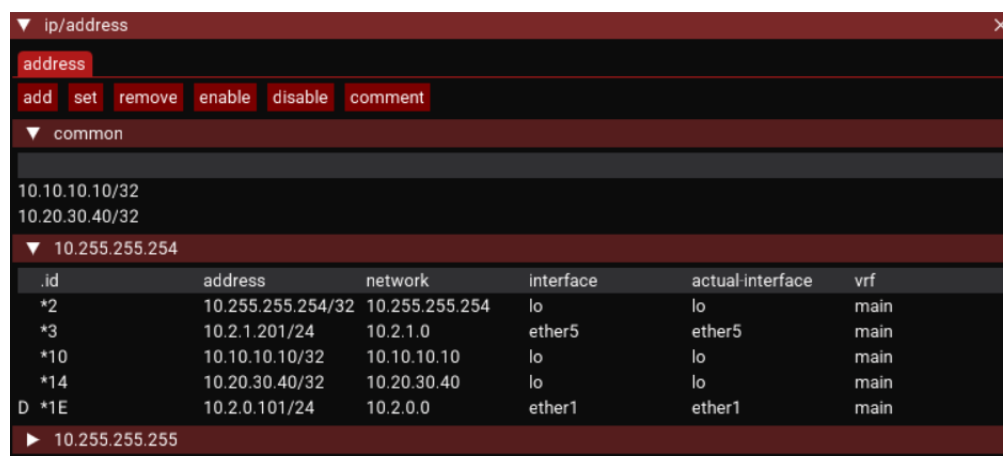
Položka *test* je výstupem z konfigurovaných zařízení. Jedná se o indikátor správnosti zadaných parametrů. Kontrola probíhá až po kliknutí na tlačítko *apply*.

Po stisku tlačítka *apply* dojde k postupnému navázání spojení s vybranými zaříze-

ními a provedení konfiguračních změn.



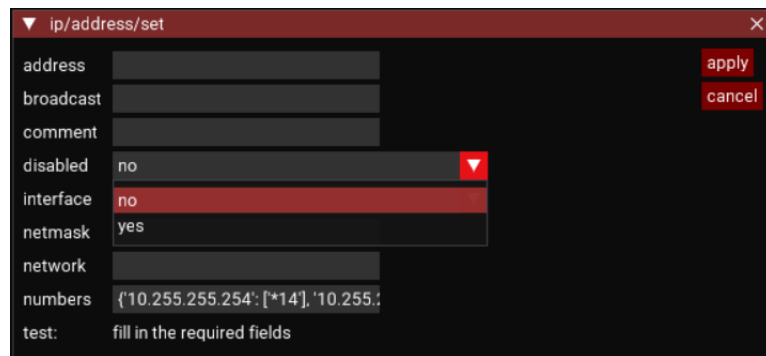
Obr. 7.3: Detail grafického rozhraní konfigurační aplikace - sekce *common*



Obr. 7.4: Detail grafického rozhraní konfigurační aplikace - záznamy vybraného zařízení

7.2.1 Vnitřní struktura

Aplikace je spouštěna jako podproces úvodní aplikace. Spouštění je provedeno pomocí příkazu *py*, za kterým následuje cesta k souboru aplikace *conf_gui.py*. Jako další spouštěcí parametr je cesta k databázovému souboru s příkazy. Jako poslední je slovník ve formě vnořených slovníků $\{ip_adresa:\{u\mathit{z}\mathit{i}\mathit{v}\mathit{a}\mathit{t}\mathit{e}\mathit{l}\mathit{s}\mathit{k}\mathit{e}_j\mathit{m}\mathit{e}\mathit{n}\mathit{o}}:\mathit{h}\mathit{e}\mathit{s}\mathit{l}\mathit{o}}\}$. Je tedy zřejmé a bylo to také testováno, že se jedná o samostatně spustitelnou aplikaci.



Obr. 7.5: Detail grafického rozhraní konfigurační aplikace - konfigurační podokno

Třída *device*

Jedná se o třídu reprezentující jednotlivá zařízení. Mezi její základní vlastnosti patří například objekt *ApiRos* zprostředkovávající surovou komunikaci na úrovni socketu či objekt *Api* sloužící ke zprostředkování samotné komunikace pomocí prostého volání funkcí.

Třída *middleware*

Okamžitě po spuštění aplikace *conf_gui* (7.2) dochází k vytvoření instance objektu *middleware*, který slouží jako adaptační vrstva mezi grafickým rozhraním a zařízeníem a databází. Tvoří tedy jednotný bod pro komunikaci se zařízeníem.

Okamžitě při inicializaci objektu *middleware* dojde k vytvoření objektu třídy *device* (7.2.1) pro každé zařízení, které se nacházelo v parametrech příkazu při spuštění. Třída *middleware* udržuje seznam objektů typu *device* po celou dobu běhu a vykonává nad nimi požadované operace. Tato třída je také zodpovědná za třídění společných prvků vícero zařízení či provedení konfiguračních změn napříč vybranými zařízeními (7.2).

Existence této třídy zajišťuje nezávislost přístupu a práce se zařízeníem. Díky tomu je možné po úpravách vytvořit například webovou aplikaci, u které by konfigurace probíhala v okně webového prohlížeče a komunikaci se zařízeníem by zajišťoval centrální server, který by mohl být snadno obohacen o další funkce, jako je například integrace adresářových služeb či snazší správa zpřístupněných uživatelských příkazů.

Výpis 7.2: Metoda objektu *middleware* (7.2.1) sloužící k provedení konfiguračních změn

```
1  def applyToDevices(self, argVals="", dirId="",
2      cmdName="", spacer="/", selected=False):
3      """Checks values and applies changes"""
4      #vypíše úplnou cestu příkazu
5      pathDef=spacer+self.printDirPath(dirId=dirId,
6          spacer=spacer)+spacer+cmdName
7      msgs=dict()
8      #postupně prochází slovník zařízení a změn, které
9      se mají provést
10     for devIp, ids in selected.items():
11         # najde správné zařízení v poli zařízení
12         for r in self.devList:
13             if r.getDevIp()==devIp:
14                 dev=r
15                 break
16         argVals=argVals.copy()
17         # řeší problém vzniklý různými způsoby výběru
18         konfigurovaných zařízení
19         if "interface" not in argVals.keys():
20             argVals["numbers"]=",".join(ids)
21         try:
22             msg = dev.api.checkValues(argVals=argVals,
23                 pathDef=pathDef) # provádí samotnou
24                 konfiguraci
25         except:
26             raise
27         if msg and "message" in msg:
28             msgs[devIp]=msg["message"]
29         else:
30             msgs[devIp]="ok"
31         # vrací výsledek všech operací (hláška "ok" či
32         chybu)
33     return msgs
```

Třída window

Jedná se o třídu reprezentující okno v programu. Slouží k uchování informací o okně, jako je například jeho název, aktuální id adresáře ve struktuře příkazů a udržuje seznam označených položek.

Závěr

Shrnutí studentské práce.

Literatura

- [1] IEEE. *The world's largest technical professional organization dedicated to advancing technology for the benefit of humanity*. Online. <https://www.ieee.org>. [cit. 2026-03-04].
- [2] Cisco Systems *AI Infrastructure, Secure Networking, and Software Solutions* Online. <https://www.cisco.com/site/us/en/index.html>. [cit. 2026-03-04].
- [3] HP *Laptop Computers, Desktops, Printers, Ink & Toner / HP® Official Site* Online. <https://www.hp.com/us-en/home.html>. [cit. 2026-03-04].
- [4] HP Enterprise *Hewlett Packard Enterprise (HPE) / Czechia* Online. <https://www.hpe.com/cz/en/home.html>. [cit. 2026-03-04].
- [5] Arista Networks *Data-Driven Cloud Networking - Arista* Online. <https://www.arista.com/en/>. [cit. 2026-03-04].
- [6] Ubiquiti Networks *Ubiquiti - Rethinking IT - Ubiquiti* Online. <https://ui.com/>. [cit. 2026-03-04].
- [7] SIA Mikrotīks *MikroTik Routers and Wireless* Online. <https://mikrotik.com/>. [cit. 2026-03-04].
- [8] MikroTik Routers and Wireless *Products* Online. <https://mikrotik.com/products>. [cit. 2026-03-04].
- [9] MikroTik Routers and Wireless *Software* Online. <https://mikrotik.com/download>. [cit. 2026-03-04].
- [10] X.200 : Information technology - Open Systems Interconnection *Basic Reference Model: The basic model* Online. <https://www.itu.int/rec/T-REC-X.200/en/>. [cit. 2026-03-04].
- [11] RouterOS - MikroTik Documentation *Python3 Example* Online. <https://help.mikrotik.com/docs/spaces/ROS/pages/47579209/Python3+Example>. [cit. 2026-03-04].
- [12] SQLite *Home Page* Online. <https://sqlite.org/>. [cit. 2026-03-04].
- [13] sqlite3 — DB-API 2.0 interface for SQLite databases *Python 3.14.0 documentation* Online. <https://docs.python.org/3/library/sqlite3.html>. [cit. 2026-03-04].

- [14] Fernet (symmetric encryption) *Cryptography 47.0.0.dev1 documentation* Online. <https://cryptography.io/en/latest/fernet/>. [cit. 2026-03-04].
- [15] Národní úřad pro kybernetickou a informační bezpečnost *Doporučení v oblasti kryptografických prostředků* Online. https://nukib.gov.cz/download/uredni_deska/Minimalni_pozadavky_v4_FINAL.pdf. [cit. 2026-03-04].
- [16] GuiProgramming - Python Wiki *GUI Programming in Python* Online. <https://wiki.python.org/moin/GuiProgramming>. [cit. 2026-03-04].
- [17] GitHub - hoffstadt/DearPyGui *Dear PyGui: A fast and powerful Graphical User Interface Toolkit for Python with minimal dependencies* Online. <https://github.com/hoffstadt/DearPyGui>. [cit. 2026-03-04].
- [18] Dear PyGui's Documentation *Dear PyGui documentation* Online. <https://dearpygui.readthedocs.io/en/latest/index.html>. [cit. 2026-03-04].
- [19] Cisco Systems *Role-Based CLI Access* Online. https://www.cisco.com/en/US/docs/ios/12_3t/12_3t7/feature/guide/gtclivws.html. [cit. 2026-03-04].
- [20] Zabbix *The enterprise-class open source observability solution* Online. <https://www.zabbix.com>. [cit. 2026-05-25].
- [21] LibreNMS Online. <https://www.librenms.org>. [cit. 2026-05-25].

Seznam symbolů a zkratek

Šířka levého sloupce Seznamu symbolů a zkratek je určena šířkou parametru prostředí `acronym` (viz řádek 1 výpisu zdrojáku na str. ??)

KolikMista pouze ukázka vyhrazeného místa

DSP číslicové zpracování signálů – Digital Signal Processing

f_{vz} vzorkovací kmitočet

Seznam příloh